# Poster: Efficient AES-GCM Decryption Under Homomorphic Encryption


Ehud Aharoni
IBM Research
Israel

Nir Drucker
IBM Research
Israel

Gilad Ezov
IBM Research
Israel

Eyal Kushnir
IBM Research
Israel

Hayim Shaul
IBM Research
Israel

Omri Soceanu
IBM Research
Israel

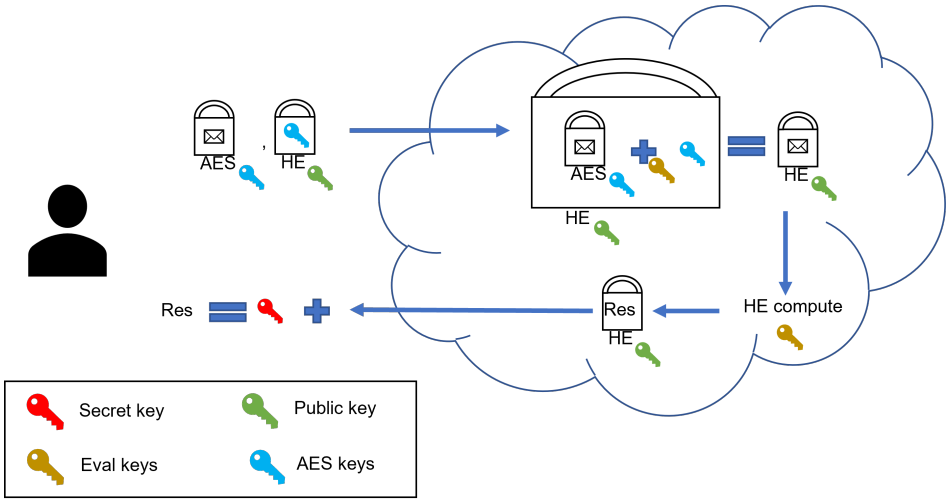




**Figure 1: A Hybrid encryption a.k.a. transciphering flow. A user possesses a symmetric AES key (blue) and an HE secret key (red). They upload the HE public key (green) and the evaluation key (brown) to the cloud. The user encrypts the data using AES and sends it to the cloud, along with an encryption of the AES keys under HE. The cloud replaces the AES encryption with HE encryption, processes the data, and returns the results FHE encrypted back to the user. Finally, the user utilizes the secret HE key to decrypt the results.**


## ABSTRACT

Computation delegation to untrusted third-party while maintaining data confidentiality is possible with homomorphic encryption (HE). However, in many cases, the data was encrypted using another cryptographic scheme such as AES-GCM. Hybrid encryption (a.k.a Transciphering) is a technique that allows moving between cryptosystems, which currently has two main drawbacks: 1) lack of standardization or bad performance of symmetric decryption under FHE; 2) lack of input data integrity.

We report the first implementations of AES-GCM decryption under CKKS, which is the fastest implementation of standardized and commonly used symmetric encryption under homomorphic encryption that also provides integrity. Our solution opens the door to end-to-end implementations such as encrypted deep neural networks while relying on AES-GCM encrypted input.



Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).


*CCS '23, November 26–30, 2023, Copenhagen, Denmark*


© 2023 Copyright held by the owner/author(s).


ACM ISBN 979-8-4007-0050-7/23/11.
https://doi.org/10.1145/3576915.3624377



## CCS CONCEPTS

• **Theory of computation** → *Computational complexity and cryptography*; • **Security and privacy** → **Symmetric cryptography and hash functions**; **Public key encryption**; • **Information systems** → *Cloud based storage*; **Data encryption**;

## KEYWORDS

homomorphic encryption, hybrid encryption, transciphering, standard implementations, AES256-GCM

**ACM Reference Format:**
Ehud Aharoni, Nir Drucker, Gilad Ezov, Eyal Kushnir, Hayim Shaul, and Omri Soceanu. 2023. Poster: Efficient AES-GCM Decryption Under Homomorphic Encryption. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS '23), November 26–30, 2023, Copenhagen, Denmark.* ACM, New York, NY, USA, 3 pages. https://doi.org/10.1145/3576915.3624377

# 1 INTRODUCTION

One trend nowadays for organizations is to shift their operations from in-house data centers to untrusted environments such as the cloud. Even traditional sectors like finance and healthcare have embraced this shift. Nevertheless, the adoption of these external services can be limited by the need to adhere to government regulations such as GDPR.

To increase the trust given by organizations in the cloud, modern cryptography offers some effective and standardized methods to secure the organization data during transmission or storage. For instance, TLS 1.3 provides security during data transit, while cryptographic primitives such as advanced encryption standard (AES)-Galois / counter mode (GCM) provide confidentiality and integrity of data at rest. However, when it comes to utilizing the cloud's computing power, a challenge arises and there is a need for heavier cryptographic tools. One cryptographic method that recently gains more and more popularity is homomorphic encryption (HE), which permits computations to be executed on encrypted data.

Interestingly, HE-based literature often concentrates on demonstrating the applicability of HE. For instance, it is frequently leveraged for conducting deep neural network inference operations as in [11] and [3], which often assume an ephemeral application in which the data is encrypted using HE, uploaded to an untrusted environment for HE computation, and the results are returned to the user for decryption.

However, in practice, the situation is more complex because users store their encrypted data and use it for cloud-based computations at different places and times. This can be an issue when using HE because storing HE-encrypted data has a compression ratio of more than 2:1 when compared to storing the original plaintext, or to the 1:1 compression ratio achieved with symmetric encryption methods like AES. This expansion ratio increases the data storage costs and also the bandwidth usage costs for transmitting the data between the storage location and the compute units.

Hybrid encryption, a.k.a., transciphering (see e.g., [12]), enables the encryption of data using symmetric block ciphers, which can then be transmitted and stored in the cloud at moderate costs. Subsequently, the encrypted data can be moved to a computing service that employs HE. Through transciphering, the service effectively transforms the encryption scheme from symmetric encryption to HE encryption. Once the data is encrypted using HE, the service can perform computations on it and return the results to the user or store them for future use. Figure 1 illustrates this process.

Till our work, there were three main drawbacks to hybrid encryption: a) it is either considered unpractical for many applications; b) it involves non-standardized symmetric ciphers, which again places some restrictions on organizations from using such a solution; c) it did not consider standardized authenticated encryption with associated datas (AEADs) schemes.

**Our contribution.** Our contribution is as follows:

- We provide the first hybrid encryption implementation that can be considered practical by many applications that use standardized block-cipher, specifically AES-CTR. We do that by using the IBM® HElayers library [1] compiled with the CryptoLab® HEaaN [8] library and running on a commodity GPU. Our implementation runs in 4 minutes for 512KB of data, with amortized latency of 7 msec per AES block.
- Today, many applications attempt to ensure both the confidentiality and the integrity of the user data. To this end, using an AEAD scheme is advantageous over using solely a symmetric cipher. Consequently, we have enhanced our implementation to include the novel integration of hybrid encryption with a standardized AEAD, specifically AES-GCM. Our implementation exhibits an efficient performance, processing 512KB of data in a mere 11.5 minutes, with an amortized latency of 21 milliseconds per AES block.

**Table 1: Standardized AES-ECB implementations under FHE, Reported measurements are of the cited references, which may derived using different hardware.**

| Ref. | Scheme | Security bits | Latency (hours) | Amortized latency (min/AES block) |
|---|---|---|---|---|
| [10] | BGV | 128 | 65 | 5 |
| [10] | BGV | 128 | 36 | 40 |
| [14] | BGV | 128 | 0.023 | 1.4 |
| [4] | DGHV | 72 | 113 | 12 |
| [7] | DGHV | 72 | 18.3 | 33 |
| [7] | DGHV | 72 | 3.58 | 0.38 |
| [7] | DGHV | 80 | 102 | 3.25 |
| [13] | TFHE | 128 | 0.07 | 4.2 |

**Related work.** Related work for transciphers involve two principle categories: a) implementations of transciphers that use standard symmetric encryption e.g., AES-electronic codebook (ECB); b) implementations of transciphers that offer a new non-standardized symmetric encryption schemes.

The first type of implementations is compared in Table 1. Some implementations offer less than 128-bit security e.g., [4, 7], while others took over an hour to complete e.g., [10, 14]. Among these implementations, the one with the fastest amortized latency was [7] that used the BGV HE scheme, which took 0.38 minutes per **AES-128** block while the FHE security level was only 72 bits. In contrast, our implementation is several orders of magnitude faster, taking only 7 amortized milli-seconds to decrypt an **AES-256** block when decrypting a batch of 32,768 AES blocks, using HE configuration of 128 bits security. We consider amortized latency instead of latency, because the input to modern tasks is often large, e.g., 602 KB of data are required for just one image sample from the commonly used ImageNet dataset. This means that we need to consider the decryption latency of 37,632 AES blocks and not just the latency of a single AES block.

The second type of implementations is summarized in Figure 2. None of these schemes are yet standardized, making them unsuitable for our case. Additionally, comparing their performance to the standardized scheme would be meaningless as it would not be an apples-to-apples comparison.

**Our implementation.** To achieve high throughput we decided to leverage the approximated HE scheme CKKS [5]. Moreover, CKKS

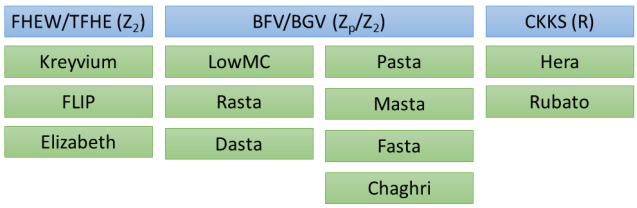



**Figure 2: Non-standardized block cipher proposals. Each block cipher is described below the HE scheme that it targets. The order in which the block ciphers is listed is arbitrary.**

**Table 2: A comparison of decryption methods under HE**

| Cipher | Process unit | Size (KB) | Latency (min) | Amortized latency (mSec/block) | Peak memory (GB) |
|---|---|---|---|---|---|
| CTR | CPU | 512 | 31 | 56.7 | 127.62 |
| CTR | GPU | 512 | 4 | 7.3 | 60.10 |
| GCM | GPU | 512 | 11.4 | 21 | 65.10 |

is the leading HE scheme when considering state-of-the-art inference applications, for example, [3, 11]. For that, we leveraged a recent technique called BLEACH [9], which has demonstrated the practicality of executing binary circuits over CKKS. Specifically, it showed that performing boolean operations on two encrypted bits, followed by the cleanup function $h_1(x) = -2x^3 + 3x^2$ [6], does not introduce any significant increase in the ciphertext error. This allows us to efficiently execute these operations while maintaining the desired level of accuracy in the ciphertext.

## 2 EXPERIMENTS

**Experimental setup.** For the experiments, we considered two platforms: a) CPU: An Intel® Xeon® CPU E5-2699 v4 @ 2.20GHz machine with 44 cores (88 threads) and 750GB memory and b) GPU: A100 SXM4 80 GB GPU, on a server with an AMD® EPYC 7763 64-Core Processor 2.45GHz machine with 64 cores (128 threads). Used single CPU thread by setting OMP_NUM_THREADS=1. The experiments were conducted using HElayers [1], a software development kit (SDK) for privacy-preserving computations that offers various programming capabilities for developers working with HE. Each experiment was repeated 10 times, and the reported result represents the minimum measured running time. In our experiments, we configured HElayers to utilize a bootstrappable HEaaN context (using the CryptoLab HEaaN library) with a security level of 128 bits. The ciphertexts employed had a capacity of 32,768 slots, allowing for parallel processing of multiple data elements. We set the multiplication depth to 12, furthermore, the fractional part precision (scale) was set to 42 bits, while the integer part precision (number of additional bits in the first prime) was set to 18 bits. In addition, the chain index after bootstrap is 12 and the minimal chain index for bootstrap is 3.

**Experiments results.** Table 2 presents[1] the benchmark results for the decryption process of various block ciphers in different modes of operation. The chosen data sizes, specifically 512KB for AES-CTR/GCM, correspond to the number of blocks required to fill the ciphertexts representing the block cipher states: 128 ciphertexts for AES-CTR/GCM. This size was selected to maximize the utilization of our implementation, as lower values would leave unused slots in the ciphertexts and result in under-utilization. It is important to note that the decryption time increases linearly with the number of $2^{15}$ blocks, as our implementation currently utilizes only one CPU thread for one GPU device. The reported latency values are given in minutes, while the amortized latency values are reported in milliseconds, which represents a significant improvement compared to the previous methods outlined in Table 1. It is evident that the fastest implementation among the tested implementations is AES-CTR, as it solely provides confidentiality guarantees. Conversely, AES-GCM offer both confidentiality and authenticity capabilities, resulting in slower performance.

[1]The AES-CTR/GCM results were also presented in a poster [2].

## REFERENCES


[1] Ehud Aharoni, Allon Adir, Moran Baruch, Nir Drucker, Gilad Ezov, Ariel Farkash, Lev Greenberg, Ramy Masalha, Guy Moshkowich, Dov Murik, et al. 2023. HElayers: A tile tensors framework for large neural networks on encrypted data. *PoPETs* (2023). https://doi.org/10.56553/popets-2023-0020

[2] Ehud Aharoni, Nir Drucker, Gilad Ezov, Eyal Kushnir, Hayim Shaul, and Omri Soceanu. 2023. E2E near-standard hybrid encryption. Poster session at 6th HomomorphicEncryption.org Standards Meeting, Seoul, South korea.

[3] Moran Baruch, Nir Drucker, Gilad Ezov, Eyal Kushnir, Jenny Lerner, Omri Soceanu, and Itamar Zimerman. 2023. Sensitive Tuning of Large Scale CNNs for E2E Secure Prediction using Homomorphic Encryption. arXiv:2304.14836 [cs.LG]

[4] Jung Hee Cheon, Jean-Sébastien Coron, Jinsu Kim, Moon Sung Lee, Tancrède Lepoint, Mehdi Tibouchi, and Aaram Yun. 2013. Batch Fully Homomorphic Encryption over the Integers. In *EUROCRYPT 2013*. Springer Berlin Heidelberg, 315–335. https://doi.org/10.1007/978-3-642-38348-9_20

[5] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. 2017. Homomorphic encryption for arithmetic of approximate numbers. In *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 409–437. https://doi.org/10.1007/978-3-319-70694-8_15

[6] Jung Hee Cheon, Dongwoo Kim, and Duhyeong Kim. 2020. Efficient homomorphic comparison methods with optimal complexity. In *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 221–256. https://doi.org/10.1007/978-3-030-64834-3_8

[7] Jean-Sébastien Coron, Tancrède Lepoint, and Mehdi Tibouchi. 2014. Scale-Invariant Fully Homomorphic Encryption over the Integers. In *Public-Key Cryptography – PKC 2014*, Hugo Krawczyk (Ed.). Springer Berlin Heidelberg, 311–328. https://doi.org/10.1007/978-3-642-54631-0_18

[8] CryptoLab. 2022. HEaaN: Homomorphic Encryption for Arithmetic of Approximate Numbers, 3.1.4. https://www.cryptolab.co.kr/eng/product/heaan.php

[9] Nir Drucker, Guy Moshkowich, Tomer Pelleg, and Hayim Shaul. 2022. BLEACH: Cleaning Errors in Discrete Computations over CKKS. Cryptology ePrint Archive, Paper 2022/1298. https://eprint.iacr.org/2022/1298

[10] Craig Gentry, Shai Halevi, and Nigel P. Smart. 2012. Homomorphic Evaluation of the AES Circuit. In *CRYPTO 2012*, Reihaneh Safavi-Naini and Ran Canetti (Eds.), Vol. 7417 LNCS. Springer Berlin Heidelberg, 850–867. https://doi.org/10.1007/978-3-642-32009-5_49

[11] Eunsang Lee, Joon-Woo Lee, Junghyun Lee, Young-Sik Kim, Yongjune Kim, Jong-Seon No, and Woosuk Choi. 2022. Low-Complexity Deep Convolutional Neural Networks on Fully Homomorphic Encryption Using Multiplexed Parallel Convolutions. In *Proceedings of the 39th International Conference on Machine Learning*, Vol. 162. PMLR, 12403–12422. https://proceedings.mlr.press/v162/lee22e.html

[12] N P Smart and F Vercauteren. 2014. Fully homomorphic SIMD operations. *Designs, Codes and Cryptography* 71, 1 (2014), 57–81. https://doi.org/10.1007/s10623-012-9720-4

[13] Roy Stracovsky, Rasoul Akhavan, and Florian Mahdavi Kerschbaum. 2022. Faster Evaluation of AES using TFHE. https://drive.google.com/file/d/1WMBjjM416BXGoiLf16gPn6q5aLt4zZqi/view FHE.org 2022, Last accessed July 2023.

[14] Dilara Toprakhisar. 2021. Behaviour of Algebraic Ciphers in Fully Homomorphic Encryption. (Aug 2021). https://research.tue.nl/nl/studentTheses/behaviour-of-algebraic-ciphers-in-fully-homomorphic-encryption